



Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

COS132 – Imperative Programming

Practical 6 Specifications

Release Date: 20-04-2026 at 06:00

Due Date: 24-04-2026 at 23:59

Late Deadline: 26-04-2026 at 23:59

Total Marks: 60

**Read the entire specification before starting
with the practical.**

Contents

1	General Instructions	3
2	Overview	3
3	Background	4
3.1	Summation Notation	4
3.2	Factorial	4
3.3	Taylor Series for Trigonometric Functions	4
3.3.1	Sine:	4
3.3.2	Cosine:	4
3.4	The Fibonacci Sequence	5
3.5	Worked Example: Computing <code>sinApprox(1.5708, 4)</code>	5
4	Your Task	5
4.1	MathUtils	6
4.1.1	Functions	6
4.2	SeriesCalc	8
4.2.1	Functions	8
4.3	SequenceOps	10
4.3.1	Functions	10
4.4	UserInterface	12
4.4.1	Functions	12
5	Implementation Details	14
6	Upload Checklist	14
7	Submission	15

1 General Instructions

- *Read the entire assignment thoroughly before you begin coding.*
- This assignment should be completed individually.
- **Every submission will be inspected with the help of dedicated plagiarism detection software.**
- Be ready to upload your assignment well before the deadline. There is a late deadline which is 48 hours after the initial deadline which has a penalty of 20% of your achieved mark. **No extensions will be granted.**
- If your code does not compile, you will be awarded a mark of 0. The output of your program will be primarily considered for marks, although internal structure may also be tested (e.g. the presence/absence of certain functions).
- Failure of your program to successfully exit will result in a mark of 0.
- Note that plagiarism is considered a very serious offence. Plagiarism will not be tolerated, and disciplinary action will be taken against offending students. Please refer to the University of Pretoria's plagiarism page at <https://portal.cs.up.ac.za/files/departmental-guide/>.
- Unless otherwise stated, the usage of C++11 or additional libraries outside of those indicated in this specification will **not** be allowed. Some of the appropriate files you submit will be overwritten during marking to ensure compliance. **Please ensure you use C++98.**
- All functions should be implemented in the corresponding `.cpp` file. No inline implementation in the header file.
- The usage of ChatGPT and other AI-related software to generate submitted code is strictly forbidden and will be considered plagiarism.
- **You may not use recursion** for any function in this practical. All repetitive computations must use loops (`for`, `while`, or `do-while`).
- **Do NOT include using namespace std; in any file.**

2 Overview

In this practical you will be exposed to **loops** (`for`, `while`, and `do-while`) and how they can be used to compute mathematical series and iterative processes. You will implement functions spread across multiple files that together form a small mathematical computation library. You must create both the **header** (`.h`) files and the **implementation** (`.cpp`) files yourself.

3 Background

Many fundamental quantities in mathematics and science cannot be computed with a single arithmetic expression. Instead, they are defined as the *limit of a series*, an infinite sum of terms whose values decrease rapidly enough that the total converges to a finite number. In practice we compute an *approximation* by summing a finite (but large) number of terms.

3.1 Summation Notation

The capital Greek letter sigma (Σ) is used as a compact way to write a sum. The general form is:

$$\sum_{i=a}^b f(i) = f(a) + f(a+1) + \cdots + f(b)$$

where i is the *index*, a is the *lower bound*, b is the *upper bound*, and $f(i)$ is the expression evaluated at each index. For example:

$$\sum_{i=1}^4 i^2 = 1^2 + 2^2 + 3^2 + 4^2 = 1 + 4 + 9 + 16 = 30$$

3.2 Factorial

The factorial of a non-negative integer n , written $n!$, is the product of all positive integers up to and including n . The capital Greek letter pi (Π) is used as a compact way to write a product:

$$n! = \prod_{i=1}^n i = 1 \times 2 \times 3 \times \cdots \times n, \quad 0! = 1$$

Factorials grow extremely rapidly: $10! = 3\,628\,800$, $15! = 1\,307\,674\,368\,000$.

3.3 Taylor Series for Trigonometric Functions

Many functions — including **sin**, **cos**, and **exp** — can be represented as an infinite polynomial called a *Taylor series*.

3.3.1 Sine:

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

3.3.2 Cosine:

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots$$

Both series converge for all real values of x expressed in radians ($\forall x \in \mathbb{R}$). With enough terms, the approximation becomes indistinguishable from the true value.

3.4 The Fibonacci Sequence

The Fibonacci sequence is defined by the recurrence relation:

$$F(n) = \begin{cases} 1 & \text{if } n = 1 \\ 1 & \text{if } n = 2 \\ F(n-1) + F(n-2) & \text{if } n \geq 3 \end{cases}$$

giving: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

3.5 Worked Example: Computing `sinApprox(1.5708, 4)`

We approximate $\sin(\frac{\pi}{2}) \approx \sin(1.5708)$ using 4 terms of the Taylor series. Each term has the form $\frac{(-1)^n x^{2n+1}}{(2n+1)!}$:

n	Exponent $2n+1$	x^{2n+1}	$(2n+1)!$	Term
0	1	$1.5708^1 = 1.5708$	$1! = 1$	+1.5708
1	3	$1.5708^3 = 3.8758$	$3! = 6$	-0.6460
2	5	$1.5708^5 = 9.5698$	$5! = 120$	+0.0797
3	7	$1.5708^7 = 23.601$	$7! = 5040$	-0.0046
Sum $\approx$				0.9999

4 Your Task

You are required to implement all functions described in this section. You must create both the **header** (.h) files and the **implementation** (.cpp) files yourself. See Section 6 for the complete list of files you must submit.

Important: Do NOT use `<cmath>` functions (`pow`, `sin`, `cos`, `exp`, etc.) anywhere in this practical. Implement all mathematical operations using loops and the functions you write.

Task	Marks
1_MathUtils	14
2_SeriesCalc	16
3_SequenceOps	14
4_UserInterface	10
Testing	6
Total	60

Table 1: Mark allocation

Note: SeriesCalc deals with the Taylor series approximation functions. SequenceOps deals with integer sequences and iterative operations.

4.1 MathUtils

This file provides the foundational mathematical helper functions used by the rest of the library. All repetitive computation **must** use a loop.

4.1.1 Functions

- `long long factorial(int n)`
 - Computes and returns $n!$ using a loop.
 - You may assume $n \geq 0$.
 - $0! = 1$.
 - *Hint:* The return type is `long long` to accommodate larger values.
 - Example: `factorial(6)` returns 720.
- `double power(double base, int exp)`
 - Computes and returns `base` raised to the non-negative integer `exp` using a loop.
 - You may assume `exp` ≥ 0 .
 - `power(x, 0)` must return 1.0 for any `x`.
 - Example: `power(3.0, 4)` returns 81.0.
- `int sumDigits(int n)`
 - Returns the sum of the decimal digits of the **positive** integer `n`.
 - Use a loop with the modulo (%) and integer division (/) operators to extract individual digits.
 - You may assume $n > 0$.
 - Example: `sumDigits(9875)` returns $9 + 8 + 7 + 5 = 29$.
 - Example: `sumDigits(100)` returns $1 + 0 + 0 = 1$.
- `int digitalRoot(int n)`
 - Repeatedly sums the digits of `n` until a single digit remains, then returns that digit.
 - If `n` is already a single digit, the function should return `n` itself.
 - *Hint:* Use `sumDigits` inside a loop that continues while $n \geq 10$.
 - You may assume $n > 0$.
 - Example: `digitalRoot(9875)` $\rightarrow$ `sumDigits(9875)=29` $\rightarrow$ `sumDigits(29)=11` $\rightarrow$ `sumDigits(11)` so the function returns 2.
- `int countDivisors(int n)`
 - Returns the number of positive integer divisors of `n` (including 1 and `n` itself).

- Use a loop that checks every integer from 1 to n .
- You may assume $n \geq 1$.
- Example: `countDivisors(12)` returns 6 (divisors are 1, 2, 3, 4, 6, 12).
- Example: `countDivisors(7)` returns 2 (divisors are 1 and 7; 7 is prime).

4.2 SeriesCalc

This file contains functions that compute approximations of mathematical functions using their Taylor series expansions. Each approximation function **must** call `factorial` and `power` from `MathUtils`.

4.2.1 Functions

- `double sinApprox(double x, int terms)`

- Approximates $\sin(x)$ using the first `terms` terms of the Taylor series:

$$\sin(x) = \sum_{n=0}^{\text{terms}-1} \frac{(-1)^n x^{2n+1}}{(2n+1)!}$$

- `x` is given in radians.
- Each iteration of the loop computes one term and adds it to a running total.
- The sign of each term alternates, starting positive ($n = 0$ is positive).
- Example: `sinApprox(1.5708, 8)` should return a value very close to 1.00000000.
- Example: `sinApprox(0.0, 5)` returns 0.00000000.

- `double cosApprox(double x, int terms)`

- Approximates $\cos(x)$ using the first `terms` terms of the Taylor series:

$$\cos(x) = \sum_{n=0}^{\text{terms}-1} \frac{(-1)^n x^{2n}}{(2n)!}$$

- `x` is given in radians.
- The first term ($n = 0$) is $\frac{x^0}{0!} = 1$. Signs alternate, starting positive.
- Example: `cosApprox(0.0, 5)` returns 1.00000000.
- Example: `cosApprox(1.5708, 8)` should return a value very close to 0.00000000.

- `double expApprox(double x, int terms)`

- Approximates e^x using the first `terms` terms of the Taylor series:

$$e^x = \sum_{n=0}^{\text{terms}-1} \frac{x^n}{n!}$$

- Note that all terms are **positive** (no sign alternation).
- Example: `expApprox(1.0, 15)` should return a value very close to 2.71828182.
- Example: `expApprox(0.0, 5)` returns 1.00000000.

- `double piApprox(int terms)`

- Approximates π using the Leibniz formula:

$$\frac{\pi}{4} = \sum_{n=0}^{\text{terms}-1} \frac{(-1)^n}{2n+1} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots$$

- The function should return 4× the partial sum above.
- *Note:* This series converges very slowly. Thousands of terms are required for a good approximation.
- Example: `piApprox(10000)` should return a value close to 3.14149265.
- Example: `piApprox(1)` returns 4.00000000.

4.3 SequenceOps

This file contains functions that operate on integer sequences and iterative numerical processes. All functions must use loops.

4.3.1 Functions

- `long long fibonacciTerm(int n)`
 - Returns the n -th Fibonacci number (1-indexed: $F(1) = 1$, $F(2) = 1$).
 - You may assume $n \geq 1$.
 - Use two variables to track the previous two terms. **Do not use recursion.**
 - Example: `fibonacciTerm(1)` returns 1.
 - Example: `fibonacciTerm(7)` returns 13.
 - Example: `fibonacciTerm(10)` returns 55.
- `long long fibonacciSum(int n)`
 - Returns the sum of the first n Fibonacci numbers.
 - You may assume $n \geq 1$.
 - *Hint:* Call `fibonacciTerm` inside a loop, or maintain a running sum alongside the Fibonacci computation.
 - Example: `fibonacciSum(5)` = $1 + 1 + 2 + 3 + 5 = 12$.
 - Example: `fibonacciSum(7)` = $1 + 1 + 2 + 3 + 5 + 8 + 13 = 33$.
- `int collatzLength(int n)`
 - The Collatz conjecture states: given any positive integer n , repeatedly apply the following rule:
$$n \leftarrow \begin{cases} n/2 & \text{if } n \text{ is even} \\ 3n + 1 & \text{if } n \text{ is odd} \end{cases}$$
until $n = 1$.
 - Return the number of steps taken to reach 1, **not** including the starting value, but **including** the final step that produces 1.
 - You may assume $n \geq 1$.
 - Example: Starting from 6: $6 \rightarrow 3 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$. That is 8 steps, so `collatzLength(6)` returns 8.
 - Example: `collatzLength(1)` returns 0.
- `long long sumOfRange(int start, int end)`
 - Returns the sum of all integers from `start` to `end` **inclusive**.

- Use a loop.
- You may assume `start ≤ end`.
- Example: `sumOfRange(1, 5) = 1 + 2 + 3 + 4 + 5 = 15`.
- Example: `sumOfRange(3, 3)` returns 3.
- `long long productOfRange(int start, int end)`
 - Returns the product of all integers from `start` to `end` **inclusive**.
 - Use a loop.
 - You may assume `start ≤ end` and all values are positive.
 - *Note:* `productOfRange(1, n)` is equivalent to `factorial(n)`, but you should implement this function independently.
 - Example: `productOfRange(2, 5) = 2 × 3 × 4 × 5 = 120`.
 - Example: `productOfRange(4, 4)` returns 4.

4.4 UserInterface

This file is responsible for all interaction with the user. It calls functions from `MathUtils`, `SeriesCalc`, and `SequenceOps` to compute results and display them to the user.

Hint: Use `std::fixed` and `std::setprecision` from the `<iomanip>` library to control the number of decimal places displayed.

4.4.1 Functions

- `void displayFactorial()`

- Prompts the user to enter a non-negative integer n using the exact message:

```
Enter n for factorial:
```

- Reads the integer, calls `factorial(n)`, and prints the result using the exact format (followed by a newline):

```
n! = <result>
```

where `<result>` is the computed factorial value.

- Example interaction (user input shown after the prompt):

```
Enter n for factorial:
6
6! = 720
```

- `void displaySinApprox()`

- Prompts the user for a value x (in radians) and the number of `terms`, using the exact messages:

```
Enter x (radians):
Enter number of terms:
```

- Calls `sinApprox(x, terms)` and prints the result using the exact format (8 decimal places, followed by a newline):

```
sin(<x>) ~ <result>
```

where `<x>` is printed to 4 decimal places and `<result>` to 8 decimal places.

- Example:

```
Enter x (radians):
1.5708
Enter number of terms:
8
sin(1.5708) ~ 1.00000000
```

- `void displayCosApprox()`

- Prompts the user for `x` (radians) and `terms`, using the exact messages:

```
Enter x (radians):  
Enter number of terms:
```

- Calls `cosApprox(x, terms)` and prints using the same format and precision as `displaySinApprox`, but with `cos` (followed by a newline):

```
cos(<x>) ~ <result>
```

- Example:

```
Enter x (radians):  
0.0  
Enter number of terms:  
5  
cos(0.0000) ~ 1.00000000
```

- `void displayFibonacci()`

- Prompts the user for a positive integer `n` using the exact message:

```
Enter n for Fibonacci:
```

- Calls `fibonacciTerm(n)` and `fibonacciSum(n)`, then prints both results using the exact format (each followed by a newline):

```
F(<n>) = <term>  
Sum of first <n> Fibonacci numbers = <sum>
```

- Example:

```
Enter n for Fibonacci:  
7  
F(7) = 13  
Sum of first 7 Fibonacci numbers = 33
```

- `void displayCollatz()`

- Prompts the user for a positive integer `n` using the exact message:

```
Enter starting number for Collatz:
```

- Calls `collatzLength(n)` and prints using the exact format (followed by a newline):

```
Collatz sequence from <n> takes <steps> steps to reach 1.
```

- Example:

```
Enter starting number for Collatz:  
6  
Collatz sequence from 6 takes 8 steps to reach 1.
```

5 Implementation Details

- You must implement the functions **exactly** as stipulated. Failure to do so will result in compilation errors on FitchFork.
- You may only use **C++98**.
- You may only use the following standard libraries:
 - `string`
 - `iostream`
 - `iomanip`
- **Do NOT include using namespace std; in any file.**
- Do **not** use `<cmath>` functions (`pow`, `sin`, `cos`, `exp`, etc.). Implement all mathematical operations using loops and the functions you write.
- You may not use recursion anywhere in this practical.
- Compile all files together using:

```
g++ -Wall -Werror -std=c++98 -g *.cpp -o main
```

- Run with:

```
./main
```

6 Upload Checklist

The following C++ files should be placed in a zip archive named `uXXXXXXXX.zip` where `XXXXXXXX` is your student number:

- `MathUtils.cpp`
- `MathUtils.h`
- `SeriesCalc.cpp`
- `SeriesCalc.h`
- `SequenceOps.cpp`
- `SequenceOps.h`
- `UserInterface.cpp`
- `UserInterface.h`

The files must be in the **root directory** of the zip archive. When you open the zip you should immediately see your files (not a folder). If unsure how to create an archive, review the relevant tutorial slides.

7 Submission

Submit your source files on the FitchFork website (<https://ff.cs.up.ac.za/>). All functions must be implemented (or at least stubbed) before submission.

Your code must compile with:

```
g++ -Wall -Werror -std=c++98 -g *.cpp -o main
```

and run with:

```
./main
```

Remember that your `.h` files will be overwritten during marking — ensure your function signatures match the specification exactly.

You have **5 submissions** and your best submission's mark will be your final mark. Submit to the Practical 6 slot on FitchFork before the deadline. **No late submissions will be accepted after the late deadline.**